

Санкт-Петербургский государственный университет

Кафедра системного программирования
Группа 25.М71-мм

Чан Пэнсян

**Расширение функциональности планировщика
NICA-Scheduler для распределённой
обработки данных в эксперименте VM@N**

Производственная практика (технологическая)

Уровень образования: *Магистратура*

Направление *09.04.04 «Программная инженерия»*

Научный руководитель:

Доцент кафедры вычислительной физики

Немнюгин Сергей Андреевич

ОГЛАВЛЕНИЕ

ВВЕДЕНИЕ	3
1 Цель и задачи	4
2 Обзор	5
2.1 Особенности обработки данных эксперимента VM@N	5
2.2 Программная среда ROOT и FairRoot	6
2.3 Системы пакетной обработки вычислительных кластеров	6
2.4 Технологии конфигурирования XML	8
3 Расширение и реализация ключевых функциональных возможностей	9
3.1 Автоматическое создание выходного каталога.	9
3.2 Поддержка извлечения дополнительных результирующих файлов через <get extras>	10
3.3 Функция пакетного ввода на основе JSON-файла	13
3.4 Объединение частичных результатов ROOT и автоматическое распознавание имён деревьев	15
3.5 Реализация смешанного последовательного выполнения <command> и <macro>	20
3.6 Реализация ограничения количества заданий в очереди и размера массива заданий	24
Заключение	27
Список литературы	29
Приложение А	31
Приложение В	32
Приложение С	32
Приложение D	33
Приложение E	35

ВВЕДЕНИЕ

BM@N является первым экспериментом проекта NICA и направлен на исследование ядерной материи и сильного взаимодействия в условиях фиксированной мишени. Экспериментальные данные проходят несколько этапов обработки, включая моделирование, реконструкцию, калибровку и физический анализ. Как правило, программы обработки считывают набор ROOT-файлов, выполняют вычисления для заданного диапазона событий, а затем формируют новые ROOT-файлы, журналы и вспомогательные результаты. С увеличением количества входных файлов последовательная обработка на одном компьютере уже не позволяет выполнить все задачи за приемлемое время, поэтому возникает необходимость в параллельной обработке данных с использованием вычислительных ресурсов кластеров [1,14,15].

Вычислительные кластеры обычно принимают задания через системы пакетной обработки. Такие системы отвечают за формирование очередей, распределение ресурсов процессора и оперативной памяти, запуск заданий и регистрацию их состояния. Вычислительные платформы, доступные для эксперимента BM@N, не являются полностью однородными: среды HybriLIT, Govogun, CICC и NCX могут использовать различные системы управления заданиями и политики распределения ресурсов. Если пользователь самостоятельно подготавливает сценарии sbatch или qsub для каждой платформы, ему необходимо учитывать различия в синтаксисе параметров, механизмах массивов заданий и ограничениях очередей. Кроме того, при переносе задачи на другую платформу требуется соответствующим образом изменять сценарии запуска [2,9,14].

NICA-Scheduler создаёт единый уровень описания между пользовательскими задачами и базовыми системами пакетной обработки. Пользователь задаёт в XML-файле путь к макросу, входные и выходные файлы, диапазон обрабатываемых событий, режим запуска и параметры вычислительных ресурсов, после чего планировщик формирует локальные

команды или сценарии кластерных заданий. Такой подход позволяет сосредоточить платформозависимые детали внутри планировщика, благодаря чему пользователю необходимо поддерживать главным образом конфигурацию, непосредственно связанную с задачей физической обработки данных. Согласно существующему руководству пользователя, система поддерживает ROOT и программные комплексы на базе FairRoot, три режима запуска — local, global и container, а также системы пакетной обработки SLURM, SGE и Torque [2,3].

1 Цель и задачи

Цель практики — дальнейшая разработка системы NICA-Scheduler. В рамках реализации проекта были поставлены следующие задачи:

1. Создавать все каталоги в пути к выходному файлу, если они не существуют (используя команду `mkdir -p`). Перед началом работы проверить, возникает ли ошибка, если каталог для выходных файлов не существует.
2. Добавить опцию `extras=FILE_PATH` в тег `<get>` для получения дополнительных файлов результатов из промежуточного хранилища. `FILE_PATH` может включать регулярные выражения.
3. Реализовать возможность принимать список файлов для обработки в качестве входного JSON-файла. Список должен задаваться новой опцией `json_input` тега `<file>`. Это аналогично опции `file_input`, но работает не с текстовым файлом, содержащим элементы на отдельных строках, а со списком файлов в формате JSON.
4. Проверить все опции слияния (`merge`) для объединения частичных файлов результатов. Реализовать автоматическое определение имени дерева (`tree name`), хранящегося в выходных файлах, для конструктора `TChain` вместо фиксированного значения (`"bmndata"`).
5. Реализовать возможность добавлять тег `<command>` в стек заданий `<job>`, включающий набор макросов. В настоящее время тег

<command> может быть только одной командой в задании, но он должен чередоваться с тегами <macro>.

6. Новые опции `queuelimit=NNN` и `arraylimit=NNN` для тега <run>, чтобы преодолеть ограничения на максимальное количество заданий, которые пользователь может отправить в очередь, и ограничение на размер массива подзадач соответственно. Это может быть разработано, например, путем создания основного shell-скрипта с циклом или путем реализации непрерывного мониторинга в фоновом процессе, который отслеживает текущие параметры выполнения пользовательских задач и запускает следующую порцию задач.

2 Обзор

2.1 Особенности обработки данных эксперимента VM@N

Обработка данных экспериментов по столкновению частиц обычно разбивается по файлам и диапазонам событий. Входной файл может обрабатываться целиком либо разделяться на несколько подзадач с помощью параметров `start_event` и `event_count`. Каждая подзадача выполняет одну и ту же программу, но использует другой входной файл, выходной файл или диапазон событий. Такая организация хорошо соответствует массивам заданий: основная часть скрипта остаётся неизменной, а индекс массива выбирает соответствующий набор параметров.

Помимо основного ROOT-файла, результаты обработки могут включать журналы выполнения, выходные данные симулятора, калибровочную информацию и отладочные файлы. Основной результат, как правило, необходимо скопировать из временного высокопроизводительного хранилища обратно в общий каталог, тогда как сохранение дополнительных результатов определяется конкретной задачей пользователя. Чтобы корректно организовать операции `put`, `get` и `clean`, планировщик при формировании скрипта должен

различать основные входные и выходные данные, дополнительные файлы и временные файлы[2,5,14].

2.2 Программная среда ROOT и FairRoot

ROOT — широко используемая в физике высоких энергий платформа анализа данных, предоставляющая средства интерпретации и выполнения кода C++, файлового хранения, древовидных структур и статистического анализа. TTree хранит данные событий в колоночной форме; дерево состоит из ветвей и записей, что позволяет считывать только переменные, необходимые для анализа. Согласно официальной документации ROOT, TChain позволяет рассматривать объекты TTree с одинаковым именем из нескольких файлов как одно непрерывное дерево. Поэтому при создании TChain необходимо указать правильное имя дерева [6,7].

FairRoot предоставляет поверх ROOT инфраструктуру моделирования и анализа, используемую в программном обеспечении физических экспериментов. В этой среде BmnRoot позволяет организовывать макросы и модули обработки данных. NICA-Scheduler не реализует физические алгоритмы непосредственно, а подготавливает для этих макросов среду выполнения, параметры, а также пути входных и выходных данных.

2.3 Системы пакетной обработки вычислительных кластеров

Таблица 2-1. Абстракция систем пакетной обработки, используемая в работе

Система	Типичный способ отправки	Возможности, относящиеся к работе	Различия, скрываемые системой
---------	--------------------------	-----------------------------------	-------------------------------

Система	Типичный способ отправки	Возможности, относящиеся к работе	Различия, скрываемые системой
SLURM	sbatch	Задания, очереди, QOS, массивы заданий	Формат директив, индексы и размер массива
SGE	qsub	Очереди, выбор узлов, задачи массива	Параметры ресурсов и переменные окружения
Torque	qsub	Управление ресурсами и очередями в стиле PBS	Заголовок скрипта и идентификаторы заданий

NICA-Scheduler преобразует перечисленные различия в единый набор параметров элемента `run`. Параметр `mode` задаёт локальный запуск, непосредственный запуск через систему пакетной обработки или запуск в контейнере; `count` управляет числом параллельных задач; `config` указывает среду, которую необходимо загрузить перед запуском; параметры `queue`, `qos`, `hosts`, `priority`, `memory1` и другие дополнительно описывают требования к ресурсам. Пользователь применяет одну и ту же структуру XML, а модуль генерации скриптов формирует синтаксис, соответствующий значениям `batch` и `mode`.

Массивы заданий SLURM служат непосредственной основой для реализованного в работе параметра `arraylimit`. Задачи внутри массива используют общий скрипт, а выбор параметров выполняется с помощью

переменной SLURM_ARRAY_TASK_ID и аналогичных переменных. Диапазон индексов массива и число одновременно выполняемых задач могут задаваться параметрами отправки. Параметр arraylimit ограничивает количество подзадач в одном массиве, тогда как queuelimit определяет число заданий, которые пользователь может отправить или удерживать в очереди. Эти ограничения не являются взаимозаменяемыми [9].

2.4 Технологии конфигурирования XML

XML подходит для описания задач с иерархической структурой. В NICA-Scheduler базовой единицей является job, который может содержать элементы macro, command и run. Внутри macro элемент file описывает входные и выходные данные, а также диапазон событий; элементы put, get и clean задают операции с временным хранилищем. Имена атрибутов имеют однозначный прикладной смысл, поэтому пользователь может изменять задачу без непосредственного редактирования исходного кода на языке C++.

Таблица 2-2. Основные XML-элементы NICA Scheduler

Элемент	Назначение	Типичное содержимое
job	Определение планируемой задачи и зависимостей	name batch dependency
macro	Определение выполняемого макроса ROOT	path start_event event_count add_args
file	Определение входных и выходных файлов	input file_input json_input output merge

Элемент	Назначение	Типичное содержимое
put	Копирование входных данных во временное хранилище	command directory
get	Извлечение результатов и дополнительных файлов	command directory extras
clean	Перечень вспомогательных файлов для удаления	path
command	Выполнение произвольной команды вне ROOT	line
run	Настройка режима и ограничений ресурсов	mode count config queuelimit arraylimit

3 Расширение и реализация ключевых функциональных возможностей

3.1 Автоматическое создание выходного каталога.

Когда output указывает на ещё не существующий многоуровневый каталог, обрабатывающий макрос не может создать файл результата, и последующие операции get или merge также не имеют допустимой цели. Эта ошибка не связана с самим макросом ROOT; причина в том, что планировщик передаёт только полное имя выходного файла, не обеспечивая существование его родительского каталога. Ручное создание каталога пользователем позволяет обойти проблему, но лишает XML-задачу самодостаточности.

Реализация сначала с помощью логики обработки путей типа `get_directory_path(output)` получает родительский каталог выходного файла. Если каталог пуст или обозначает текущий каталог, обработка не требуется; в противном случае перед выполнением задачи формируется команда `mkdir -p`. Опция `-p` позволяет рекурсивно создавать родительские каталоги при их отсутствии и обеспечивает идемпотентное поведение, если каталог уже существует [13].

Создание каталога должно происходить до команды `ROOT` и копирования результатов. В режиме `local` команда создания вызывается локальным процессом выполнения; в режимах `global` и `container` она записывается в `job`-скрипт. Таким образом, к моменту начала обработки на вычислительном узле целевой каталог уже виден. Если `output` находится в общем хранилище, путь после создания каталога также доступен последующим узлам.

3.2 Поддержка извлечения дополнительных результирующих файлов через `<get extras>`

Исходный тег `<get>` в `NICA-Scheduler` использовался для копирования основного выходного файла из каталога промежуточного хранилища вычислительного узла в указанный пользователем конечный каталог после завершения задачи. В реальных задачах обработки данных `BM@N`, помимо основного выходного файла `ROOT`, обычно также создаются журналы выполнения, отладочная информация, статистические результаты и файлы анализа качества. Если извлекать только основной выходной файл, эти дополнительные файлы останутся во временном каталоге, и пользователь не сможет напрямую использовать их для анализа состояния задачи.

Для решения этой проблемы в данной работе в тег `<get>` добавлен необязательный атрибут `extras`. Этот атрибут принимает имя файла или маску с подстановочными знаками, например:

```
<get command="cp" directory="/opt/bmnroot/macro/run/nica_tmp_get" extras="simulation*.log"/>
```

После завершения задачи NICA-Scheduler, помимо извлечения основного выходного файла, также находит в промежуточном каталоге все файлы, соответствующие `simulation*.log`, и копирует их в конечный каталог, где находится основной выходной файл. Если `extras` не задан, система по-прежнему обрабатывает только основной выходной файл, что обеспечивает совместимость с существующими XML-файлами задач.

В процессе реализации функции сначала был расширен тег `<get>` в NICA-Scheduler: добавлен необязательный атрибут `extras`, предназначенный для указания файлов, которые необходимо дополнительно извлечь, — журналов, статистических результатов и т. п. После разбора XML-файла Scheduler формирует путь для сопоставления дополнительных файлов на основе `directory` и `extras` и добавляет логику циклической обработки после исходной команды копирования основного выходного файла. По завершении задачи система с помощью подстановочных знаков находит подходящие файлы, проверяет их наличие, затем копирует их в каталог, где находится основной выходной файл, и удаляет временные копии из каталога промежуточного хранилища. Сгенерированные команды извлечения сохраняются в `GetArray` скрипта задания и единообразно выполняются соответствующей задачей. Чтобы избежать некорректного развёртывания переменных окружения внутри контейнера Apptainer, для каталога промежуточного хранилища используется абсолютный путь внутри контейнера, а конечный выходной каталог по-прежнему задаётся через `$VMCWORKDIR`. Данная реализация сохраняет совместимость с исходной функциональностью `<get>` и может использоваться во всех трёх режимах работы: `local`, `global` и `container` [4,10].

Тестирование в режиме `local`:

Проверка функции извлечения файлов тега `<get>` в NICA-Scheduler: после выполнения тестового макроса в каталоге промежуточного хранилища создаются один основной выходной файл `result.root` и два дополнительных

файла журналов simulation.log и simulation_debug.log. Используемые в тесте файл макроса и XML-файл конфигурации приведены в приложении А к работе.

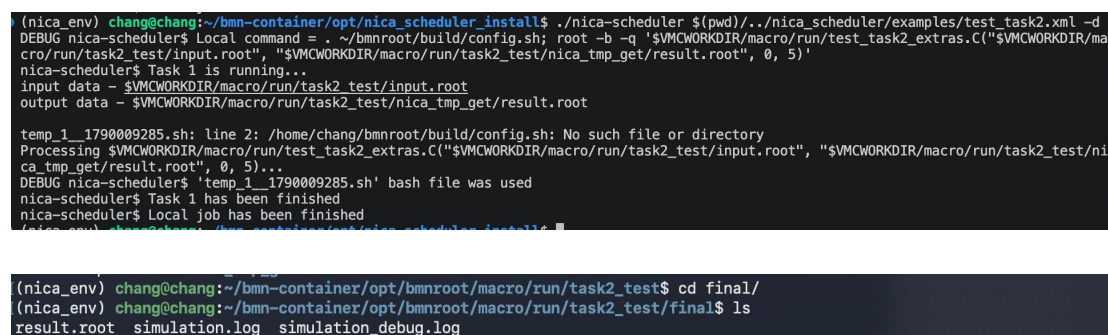
При этом directory задаёт каталог промежуточного хранилища, а extras указывает файлы журналов, которые необходимо дополнительно извлечь. После завершения выполнения задачи Scheduler должен скопировать основной выходной файл и все файлы, соответствующие шаблону simulation*.log, в конечный выходной каталог.

В корректном случае в каталоге должны находиться следующие три файла:

result.root

simulation.log

simulation_debug.log



```
(nica_env) chang@chang:~/bmn-container/opt/nica_scheduler_install$ ./nica_scheduler $(pwd)/../nica_scheduler/examples/test_task2.xml -d
DEBUG nica-scheduler$ Local command = . ~/bmnroot/build/config.sh; root -b -q '$VMCWORKDIR/macro/run/test_task2_extras.C('$VMCWORKDIR/macro/run/task2_test/input.root', '$VMCWORKDIR/macro/run/task2_test/nica_tmp_get/result.root', 0, 5)'
nica-scheduler$ Task 1 is running...
input data - $VMCWORKDIR/macro/run/task2_test/input.root
output data - $VMCWORKDIR/macro/run/task2_test/nica_tmp_get/result.root
temp_1_1790009285.sh: line 2: /home/chang/bmnroot/build/config.sh: No such file or directory
Processing $VMCWORKDIR/macro/run/test_task2_extras.C('$VMCWORKDIR/macro/run/task2_test/input.root', '$VMCWORKDIR/macro/run/task2_test/nica_tmp_get/result.root', 0, 5)...
DEBUG nica-scheduler$ 'temp_1_1790009285.sh' bash file was used
nica-scheduler$ Task 1 has been finished
nica-scheduler$ Local job has been finished
(nica_env) chang@chang:~/bmn-container/opt/nica_scheduler_install$

(nica_env) chang@chang:~/bmn-container/opt/bmnroot/macro/run/task2_test$ cd final/
(nica_env) chang@chang:~/bmn-container/opt/bmnroot/macro/run/task2_test/final$ ls
result.root simulation.log simulation_debug.log
```

Рисунок 3-1 – Результат выполнения теста <get extras>

Тестирование в режиме global и container:

В режимах global и container Scheduler генерирует скрипты заданий job_*.sh для системы пакетной обработки. Как показано на рисунке 3-2, сгенерированные в режимах global и container скрипты заданий содержат команду копирования основного выходного файла, а также цикл для сопоставления дополнительных файлов по шаблону simulation*.log. Это показывает, что атрибут extras тега <get> был корректно преобразован в операцию извлечения файлов в скрипте пакетного задания.

```

(nica_env) chang@chang:~/bmn-container/opt/nica_scheduler_install$ grep -n -E 'OutputFileArray|GetArray|CleanArray' \
job_10728670441735783011.sh
10:declare -a OutputFileArray=("$VMCWORKDIR/macro/run/task2_test/nica_tmp_get/result.root" )
20:declare -a GetArray=("$VMCWORKDIR/macro/run/task2_test/nica_tmp_get/result.root $VMCWORKDIR/macro/run/task2_test/final/result.r
oot; set +f; for extra_file in $VMCWORKDIR/macro/run/task2_test/nica_tmp_get/simulation*.log; do if [ -e \${extra_file} ]; then cp \${ex
tra_file} $VMCWORKDIR/macro/run/task2_test/final/\$(basename \${extra_file}); rm -rf \${extra_file}; fi; done; set -f" )
22:declare -a CleanArray=("$VMCWORKDIR/macro/run/task2_test/nica_tmp_get/result.root" )
53:element=${OutputFileArray[${array_index}]}
79:element=${GetArray[${array_index}]}
90:element=${CleanArray[${array_index}]}
(nica_env) chang@chang:~/bmn-container/opt/nica_scheduler_install$ grep -n -A10 -B2 'element=${GetArray}' \
job_10728670441735783011.sh
77-eval $comm
78-
79:element=${GetArray[${array_index}]}
80-if [ "${element}" != ">" ]
81-then
82- set -f # avoid globbing (expansion of *)
83-# arr=({element// / })
84-# comm="${arr[0]} ${arr[1]} ${arr[2]}"
85-# eval $comm
86- echo "DEBUG nica-scheduler: GET command = ${element}"
87- eval ${element}
88-fi
89-
(nica_env) chang@chang:~/bmn-container/opt/nica_scheduler_install$

```

Рисунок 3-2 – Фрагмент GetArray с командами копирования дополнительных файлов

3.3 Функция пакетного ввода на основе JSON-файла

Оригинальный тег <file> в NICA-Scheduler поддерживает указание одного входного файла через атрибут input, а также чтение путей к входным файлам построчно из обычного текстового файла через атрибут file_input. Однако в реальных задачах обработки данных список входных файлов также часто формируется другими программами в формате JSON. Если требуется сначала преобразовать JSON-файл в текстовый файл, чтобы передать его NICA-Scheduler, это не только добавляет дополнительный этап предварительной обработки, но и снижает совместимость системы с другими инструментами управления данными [12].

Поэтому цель данной задачи — не изменяя существующий механизм выполнения задач, добавить в тег <file> новый атрибут json_input, чтобы NICA-Scheduler мог читать несколько входных файлов из массива JSON и создавать для каждого входного файла отдельную задачу обработки данных.

Форма использования нового атрибута выглядит следующим образом:

```

<file json_input="path/to/input_files.json"

      output="path/to/out_${counter}.root"/>

```

В текущей реализации поддерживается следующий формат JSON-файла:
массив строк верхнего уровня, например:

```
[  
  "/tmp/fake_sim1.root",  
  "/tmp/fake_sim2.root"  
]
```

В данной задаче на этапе разбора XML добавлена обработка атрибута `json_input` (Приложение В). NICA-Scheduler считывает массив строк из JSON-файла и добавляет пути к файлам из него в существующий список входных файлов системы.

После завершения разбора JSON-ввод использует тот же процесс генерации задач, что и такие способы ввода, как `input` и `file_input`. Поэтому без отдельного изменения режимов `local`, `global` и `container` можно для каждого входного файла сгенерировать соответствующую команду выполнения или скрипт задачи.

В режиме `local` NICA-Scheduler успешно считал два входных файла из JSON, сгенерировал два вызова ROOT-макросов и создал соответствующие выходные файлы, что подтверждает корректность разбора JSON и процесса локального выполнения задач.

```
(nica_env) chang@chang:~/bmn-container/opt/nica_scheduler_install$ ./nica-scheduler $(pwd)/../nica_scheduler/examples/test_task3.xml -d  
DEBUG nica-scheduler$ File count in the JSON input list: 2  
DEBUG nica-scheduler$ Local command = . ~/bmnroot/build/config.sh > $VMCWORKDIR/macro/run/test_json_input/local_log 2>&1; root -b -q '$V  
MCWORKDIR/macro/run/test_json_input.C("/tmp/fake_sim1.root", "$VMCWORKDIR/macro/run/test_json_input/out_1.root", 10, 20)' > $VMCWORKDIR/  
macro/run/test_json_input/local_log 1 2>&1  
DEBUG nica-scheduler$ Local command = . ~/bmnroot/build/config.sh > $VMCWORKDIR/macro/run/test_json_input/local_log 2>&1; root -b -q '$V  
MCWORKDIR/macro/run/test_json_input.C("/tmp/fake_sim2.root", "$VMCWORKDIR/macro/run/test_json_input/out_2.root", 10, 20)' > $VMCWORKDIR/  
macro/run/test_json_input/local_log 2 2>&1  
nica-scheduler$ Task 2 is running...  
input data - /tmp/fake_sim2.root  
output data - $VMCWORKDIR/macro/run/test_json_input/out_2.root  
  
nica-scheduler$ Task 1 is running...  
input data - /tmp/fake_sim1.root  
output data - $VMCWORKDIR/macro/run/test_json_input/out_1.root  
  
DEBUG nica-scheduler$ 'temp_1_1789912272.sh' bash file was used  
nica-scheduler$ Task 1 has been finished  
DEBUG nica-scheduler$ 'temp_2_1789912272.sh' bash file was used  
nica-scheduler$ Task 2 has been finished  
nica-scheduler$ Local job has been finished
```

Рисунок 3-3 – Результат обработки списка входных файлов из JSON

В режиме `global` сгенерированный пакетный скрипт содержит корректные `InputFileArray` и `OutputFileArray`, при этом два входных файла соответствуют `out_1.root` и `out_2.root`. В режиме `container` сгенерированный скрипт также

содержит корректные входные, выходные параметры и параметры событий, а вызов ROOT-макроста записывается в команду выполнения контейнера.

```
(nica_env) chang@chang:~/bmn-container/opt/nica_scheduler_install$ cat job_687945737322302370.sh
echo "Start date: `date` (host: `hostname`)"
echo "Current directory: `pwd`"

. ~/bmnroot/build/config.sh > $VMCWORKDIR/macro/run/test_json_input/local_log 2>&1;
declare -a InputFileArray=("/tmp/fake_sim1.root" "/tmp/fake_sim2.root" )
declare -a OutputFileArray=("$VMCWORKDIR/macro/run/test_json_input/out_1.root" "$VMCWORKDIR/macro/run/test_json_input/out_2.root" )
declare -a StartEventArray=("10" "10" )
declare -a EventCountArray=("20" "20" )
declare -a AddArgsArray=(">" ">" )
declare -a PutArray=(">" ">" )
declare -a GetArray=(">" ">" )
declare -a CleanArray=()

: ${NICA_ARRAY_OFFSET:=0}

let "array_index = $SLURM_ARRAY_TASK_ID - 1 + ${NICA_ARRAY_OFFSET} + 0"
element=${PutArray[${array_index}]}
if [ "${element}" != ">" ]
then
# Check if PutElement contains "wait <file_path>"
if [[ "${element}" == *"wait "* ]]; then
# Extract file path after "wait "
```

Рисунок 3-4 – Массивы параметров в сгенерированном сценарии задания для JSON-ввода

3.4 Объединение частичных результатов ROOT и автоматическое распознавание имён деревьев

NICA-Scheduler поддерживает разделение одной задачи обработки данных на несколько подзадач, выполняемых параллельно, с помощью параметра `parallel_mode`. Каждая подзадача обрабатывает свой диапазон событий и создаёт отдельный файл частичных результатов. Например, если выходной файл задан как `out.root`, а степень параллелизма равна 4, в процессе выполнения задачи могут быть созданы следующие файлы:

```
out_1.root
out_2.root
out_3.root
out_4.root
```

После завершения всех подзадач планировщик должен выполнить постобработку этих файлов частичных результатов в соответствии с атрибутом `merge` тега `<file>` в XML-файле. NICA-Scheduler поддерживает следующие четыре способа объединения:

Значение атрибута merge	Внутренний режим	Способ обработки
false или не задано	-1	Не объединять, сохранять только файлы частичных результатов
chain	0	Создать TChain, включающий файлы частичных результатов
true	1	Объединить в один ROOT-файл и удалить файлы частичных результатов
preserve	2	Объединить в один ROOT-файл, сохранив файлы частичных результатов

В исходной реализации в режиме merge="chain" для создания TChain использовалось фиксированное имя дерева:

```
TChain chainUnion("bmndata");
```

Эта реализация по умолчанию предполагает, что дерево событий во всех ROOT-файлах результатов называется bmndata. Однако TTree, создаваемые разными макросами обработки ROOT, FairRoot или BmnRoot, могут иметь другие имена, например cbmsim, events, tree или иные пользовательские имена. Если фактическое имя дерева не bmndata, программа хотя и может создать итоговый ROOT-файл, но содержащийся в нём TChain не сможет корректно подключиться к данным событий, и в результате может получиться пустая цепочка [7].

Кроме того, некоторые ROOT-файлы могут содержать только TGeoManager, гистограммы или объекты параметров и не содержать TTree. Для таких файлов программа не должна продолжать создание недействительного TChain, а должна остановить текущую операцию объединения и вывести понятное сообщение об ошибке.

Следовательно, в данной задаче необходимо проверить логику обработки всех вариантов merge и в режиме merge="chain" автоматически определять фактическое имя TTree в файлах частичных результатов вместо фиксированного "bmndata".

Реализация:

В merge.C добавлена функция автоматического определения имени TTree. Программа открывает файлы частичных результатов, созданные параллельными задачами, перебирает ROOT-объекты в файле, находит первое допустимое TTree и использует его фактическое имя для создания TChain:

```
TString treeName = detect_tree_name_from_partial_files(files);
```

```
TChain chainUnion(treeName.Data());
```

Перед созданием TChain программа также проверяет, содержат ли все файлы частичных результатов TTree с одинаковым именем. Если какой-либо файл не удаётся открыть, в нём отсутствует целевое дерево или ни в одном из файлов нет TTree, программа выводит сообщение об ошибке и останавливает объединение, чтобы избежать создания неполного или недействительного файла результата.

Отдельное тестирование merge.C:

Чтобы проверить корректность самого merge.C, можно не запускать весь планировщик (scheduler), а вручную подготовить частичные ROOT-файлы и напрямую вызвать merge.C.

Сначала создадим тестовый макрос-файл (Приложение С) и проверим наш тестовый макрос-файл.

```
Apptainer> root -l -b -q 'create_test_tree.C("test.root","mytree",0,10)'
Processing create_test_tree.C("test.root","mytree",0,10)...
Created file: test.root, tree: mytree, entries: 10
Apptainer> root -l test.root
root [0]
Attaching file test.root as _file0...
(TFile *) 0x17364530
root [1] .ls
TFile**      test.root
TFile*       test.root
KEY: TTree   mytree;1      mytree
root [2]
```

Рисунок 3-5 – Создание тестового ROOT-файла и определение имени дерева mytree

После успешного создания тестовых файлов можно протестировать различные режимы `merge`.

Тест mode=0: создание TChain:

Для проверки функции автоматического определения имени дерева в тесте были созданы два файла частичных результатов, содержащих mytree, с числом событий 10 и 15 соответственно. Затем с использованием mode=0 был вызван merge.C; программа успешно определила имя дерева mytree и создала chain_result.root.

Результаты проверки показали, что имя TChain в выходном файле — mytree, общее число событий — 25, и он корректно ссылается на два файла частичных результатов. Оба частичных файла были сохранены, поскольку TChain по-прежнему требует доступа к данным в них.

Тестирование подтвердило, что модифицированная программа способна автоматически распознавать фактическое имя TTree, больше не зависит от фиксированного "bmndata", а создаваемый TChain имеет корректное содержимое.

```

Apptainer> root -l -b -q 'create_test_tree.C("chain_part_1.root","mytree",0,10)'
Processing create_test_tree.C("chain_part_1.root","mytree",0,10)...
Created file: chain_part_1.root, tree: mytree, entries: 10
Apptainer> root -l -b -q 'create_test_tree.C("chain_part_2.root","mytree",10,15)'
Processing create_test_tree.C("chain_part_2.root","mytree",10,15)...
Created file: chain_part_2.root, tree: mytree, entries: 15
Apptainer> root -l -b -q 'merge.C("chain_result.root chain_part_1.root chain_part_2.root",0)'
Processing merge.C("chain_result.root chain_part_1.root chain_part_2.root",0)...
Detected TTree name for TChain: "mytree" from file "chain_part_1.root"
The Chain with 25 event(s) was written to file "chain_result.root" using TTree name "mytree" and pointing to following files:
chain_part_1.root
chain_part_2.root
Apptainer>

```

Рисунок 3-6 – Автоматическое определение имени дерева и создание TChain в режиме mode=0

Тест mode=1 (режим объединения с удалением)

В тесте использовались два файла частичных результатов, содержащих 10 и 15 событий соответственно, и merge.C вызывался с mode=1. После выполнения был успешно создан итоговый файл результатов, содержащий 25 событий; оба файла частичных результатов были удалены.

```

Apptainer> root -l -b -q 'merge.C("chain_result.root chain_part_1.root chain_part_2.root",1)'
Processing merge.C("chain_result.root chain_part_1.root chain_part_2.root",1)...
[TFFile::Cp] Total 0.01 MB |=====| 100.00 % [0.9 MB/s]
[TFFile::Cp] Total 0.01 MB |=====| 100.00 % [2.5 MB/s]
[TFFile::Cp] Total 0.00 MB |=====| 100.00 % [0.2 MB/s]
Temporary files were successfully removed

```

Рисунок 3-7 – Объединение ROOT-файлов в режиме mode=1 с удалением частичных результатов

Тестирование подтвердило, что mode=1 позволяет корректно объединить данные и после успешного объединения очистить файлы частичных результатов.

Тест mode=2 (режим объединения с сохранением)

В тесте использовались те же два файла частичных результатов, и merge.C вызывался с mode=2. После выполнения был успешно создан итоговый файл результатов, содержащий 25 событий, при этом оба файла частичных результатов также сохранились.

```

Apptainer> ls
chain_result.root create_test_tree.C merge.C test.root
Apptainer> root -l -b -q 'create_test_tree.C("part_1.root","mytree",0,10)'

Processing create_test_tree.C("part_1.root","mytree",0,10)...
Created file: part_1.root, tree: mytree, entries: 10
Apptainer> root -l -b -q 'create_test_tree.C("part_2.root","mytree",10,15)'

Processing create_test_tree.C("part_2.root","mytree",10,15)...
Created file: part_2.root, tree: mytree, entries: 15
Apptainer> root -l -b -q 'merge.C("result.root part_1.root part_2.root",2)'

Processing merge.C("result.root part_1.root part_2.root",2)...
[TFFile::Cp] Total 0.01 MB      |=====| 100.00 % [2.5 MB/s]
[TFFile::Cp] Total 0.01 MB      |=====| 100.00 % [2.5 MB/s]
[TFFile::Cp] Total 0.00 MB      |=====| 100.00 % [0.2 MB/s]
Partial result files were merged and preserved
Apptainer> ls
chain_result.root create_test_tree.C merge.C part_1.root part_2.root result.root test.root

```

Рисунок 3-8 – Объединение ROOT-файлов в режиме mode=2 с сохранением частичных результатов

Тестирование подтвердило, что mode=2 позволяет корректно выполнить объединение данных и в соответствии с требованиями сохранить исходные файлы частичных результатов.

3.5 Реализация смешанного последовательного выполнения <command> и <macro>

NICA-Scheduler использует XML-файлы для описания пользовательских задач обработки данных. Исходная система поддерживала выполнение ROOT-макросов с помощью тега <macro>, а также выполнение обычных Shell-команд с помощью тега <command>. Однако эти два типа исполнительных единиц в исходной реализации были независимы друг от друга: когда в задаче присутствовал тег <command>, планировщик распознавал эту задачу как задачу, содержащую только команды, и переходил в специальную ветку обработки command-only. Следовательно, пользователь не мог одновременно настроить <command> и <macro> в одном <job> и не мог поочерёдно выполнять их в порядке записи в XML.

В реальном процессе обработки данных BM@N полная задача обычно включает не только вычисления с помощью ROOT-макросов, но и обычные команды, такие как подготовка окружения перед вычислениями, создание каталогов, запись состояния и обработка файлов после вычислений. Например, пользователю может потребоваться сначала выполнить одну Shell-команду,

затем запустить ROOT-макрос, а после этого продолжить выполнение других команд или макросов. Следовательно, цель данной задачи — поддерживать несколько тегов `<command>` и `<macro>` в одном `<job>` и гарантировать, что они выполняются строго в порядке их расположения в XML-файле.

В исходном NICA-Scheduler теги `<command>` и `<macro>` сохранялись раздельно. При этом обычные команды могли сохраняться только в одном отдельном поле, поэтому несколько команд перезаписывали друг друга; кроме того, система не могла записать относительный порядок команд и макросов в XML-файле.

Для решения этой проблемы в структуру параметров задачи была добавлена единая последовательность выполнения `vecJobStack`. При чтении каждого тега `<command>` или `<macro>` XML-парсер добавляет его в эту последовательность в порядке появления и с помощью поля типа различает обычные команды и ROOT-макросы.

На этапе выполнения задачи все три режима — `local`, `global` и `container` — были изменены на обход `vecJobStack`. Обычные команды выполняются напрямую или записываются в скрипт задачи, а макросы преобразуются в соответствующие команды вызова ROOT. Исходный контейнер `vecMacros` сохраняется и продолжает использоваться для обработки параметров макросов, связей ввода-вывода, параллельного выполнения и объединения результатов.

Для проверки того, могут ли обычные команды и ROOT-макросы выполняться поочерёдно в порядке, заданном в XML, в тесте были созданы два ROOT-макрона.

`task5_macro1.C` (Приложение D) при выполнении записывает `MACRO_1` в `order.txt` и создаёт `out_macro1.root`; `task5_macro2.C` (Приложение D) при выполнении записывает `MACRO_2` и создаёт `out_macro2.root`. Второй макрос использует выходной файл первого макрона в качестве входного, что позволяет одновременно проверить передачу файлов между макронами.

Тестовая задача настроена в следующем порядке:

COMMAND_1 → MACRO_1 → COMMAND_2 → MACRO_2 → COMMAND_3

Три обычные команды и два тестовых макроса записывают в order.txt соответствующую метку. Поэтому после завершения задачи по этому файлу можно напрямую определить, правилен ли фактический порядок выполнения.

Содержимое задач в трёх XML-файлах (Приложение D) одинаково; основное различие состоит в том, что параметр mode в теге <run> установлен соответственно в local, global и container.

Тестирование в режиме local:

Запустим тестовую задачу из в режиме отладки. После завершения задачи просмотрите содержимое order.txt и проверьте, успешно ли созданы два выходных файла ROOT. Фактически полученный порядок выполнения:

COMMAND_1

MACRO_1

COMMAND_2

MACRO_2

COMMAND_3

Этот результат полностью совпадает с порядком, определённым в XML. При этом out_macro1.root и out_macro2.root были успешно созданы, что показывает, что оба макроса были выполнены, а передача файлов между макросами работает корректно.

```
(nica_env) chang@chang:~/bmn-container/opt/bmnroot/macro/run/task5_test$ ls
input.root order.txt out_macro1.root out_macro2.root task5_macro1.C task5_macro2.C
(nica_env) chang@chang:~/bmn-container/opt/bmnroot/macro/run/task5_test$ cat order.txt
COMMAND_1
MACRO_1
COMMAND_2
MACRO_2
COMMAND_3
```

```
(nica_env) chang@chang:~/bmn-container/opt/nica_scheduler_install$ ./nica-scheduler $(pwd)/../nica_scheduler/examples/test_task5.xml -d
DEBUG nica-scheduler$ 'temp_1_1790003050.sh' bash file was used
nica-scheduler$ Task 1 has been finished
DEBUG nica-scheduler$ Local command line = . ~/bmnroot/build/config.sh; echo COMMAND_2 >> $VMCWORKDIR/macro/run/task5_test/order.txt
nica-scheduler$ Task 1 is running: command line - echo COMMAND_2 >> $VMCWORKDIR/macro/run/task5_test/order.txt
temp_command_1_3_1790003060.sh: line 1: /home/chang/bmnroot/build/config.sh: No such file or directory
DEBUG nica-scheduler$ 'temp_command_1_3_1790003060.sh' bash file was used
DEBUG nica-scheduler$ Local command = . ~/bmnroot/build/config.sh; root -b -q '$VMCWORKDIR/macro/run/task5_test/task5_macro2.C('$VMCWORKDIR/macro/run/task5_test/out_macro1.root', '$VMCWORKDIR/macro/run/task5_test/out_macro2.root', 10, 20)'
nica-scheduler$ Task 1 is running...
input data - $VMCWORKDIR/macro/run/task5_test/out_macro1.root
output data - $VMCWORKDIR/macro/run/task5_test/out_macro2.root

temp_1_1790003060.sh: line 2: /home/chang/bmnroot/build/config.sh: No such file or directory
Processing $VMCWORKDIR/macro/run/task5_test/task5_macro2.C('$VMCWORKDIR/macro/run/task5_test/out_macro1.root', '$VMCWORKDIR/macro/run/task5_test/out_macro2.root', 10, 20)...
DEBUG nica-scheduler$ 'temp_1_1790003060.sh' bash file was used
nica-scheduler$ Task 1 has been finished
DEBUG nica-scheduler$ Local command line = . ~/bmnroot/build/config.sh; echo COMMAND_3 >> $VMCWORKDIR/macro/run/task5_test/order.txt
nica-scheduler$ Task 1 is running: command line - echo COMMAND_3 >> $VMCWORKDIR/macro/run/task5_test/order.txt
temp_command_1_5_1790003064.sh: line 1: /home/chang/bmnroot/build/config.sh: No such file or directory
DEBUG nica-scheduler$ 'temp_command_1_5_1790003064.sh' bash file was used
nica-scheduler$ Local job has been finished
(nica_env) chang@chang:~/bmn-container/opt/nica_scheduler_install$
```

Рисунок 3-9 – Порядок выполнения команд и макросов в режиме local

Тестирование в режиме global и container:

В режимах global и container NICA-Scheduler формирует на основе содержимого задачи скрипт job_*.sh для системы пакетной обработки. Тест в основном проверяет порядок команд в этом скрипте.

Основное содержимое сгенерированного скрипта должно располагаться в следующем порядке:

```
echo COMMAND_1

root -b -q 'task5_macro1.C(...)'

echo COMMAND_2

root -b -q 'task5_macro2.C(...)'

echo COMMAND_3
```

Результаты проверки показывают, что порядок обычных команд и ROOT-макросов в сгенерированном скрипте совпадает с XML-файлом, что означает, что vecJobStack уже правильно применён в процессе генерации скриптов для режимов global и container.

```
(nica_env) chang@chang:~/bmn-container/opt/nica_scheduler_install$ grep -nE '^echo COMMAND_|task5_macro[12]\.C' job_2062
9822561692639399.sh
27:echo COMMAND_1 > $VMCWORKDIR/macro/run/task5_test/order.txt
55:comm="root -b -q '$VMCWORKDIR/macro/run/task5_test/task5_macro1.C('${element})'"
108:echo COMMAND_2 >> $VMCWORKDIR/macro/run/task5_test/order.txt
136:comm="root -b -q '$VMCWORKDIR/macro/run/task5_test/task5_macro2.C('${element})'"
189:echo COMMAND_3 >> $VMCWORKDIR/macro/run/task5_test/order.txt
(nica_env) chang@chang:~/bmn-container/opt/nica_scheduler_install$
```

3.6 Реализация ограничения количества заданий в очереди и размера массива заданий

При обработке данных эксперимента BM@N одна задача обычно требует обработки большого количества входных файлов. NICA-Scheduler на основе информации о файлах в конфигурации XML генерирует несколько подзадач и отправляет их в вычислительный кластер через системы пакетной обработки, такие как SLURM, SGE или Torque.

В исходной реализации все подзадачи сразу организовывались в один пакетный массив и отправлялись. Когда количество входных файлов велико, размер одного массива заданий может превысить максимально допустимый в кластере. Кроме того, разные вычислительные платформы ограничивают количество заданий одного пользователя, находящихся в очереди или выполняющихся. Если не контролировать количество отправляемых заданий, могут возникнуть такие проблемы, как сбой при отправке заданий, чрезмерная нагрузка на систему планирования, а также неполнота зависимостей задач последующего объединения.

Для решения вышеуказанной проблемы в данной задаче в тег <run> добавлены два параметра: arraylimit и queuelimit. При этом arraylimit используется для ограничения количества заданий массива, отправляемых за один раз, разбивая крупную задачу на несколько меньших пакетов; queuelimit используется для ограничения количества заданий текущего пользователя в пакетной очереди и приостанавливает отправку новых заданий при достижении лимита.

Реализация:

Сначала в структуру параметров задачи добавляются iArrayLimit и iQueueLimit, предназначенные для сохранения параметров arraylimit и

queuelimit тега <run> в XML-файле, а при чтении выполняется проверка корректности этих параметров.

Когда общее количество задач превышает arraylimit, Scheduler разбивает задачи на несколько партий. Например, при 5 задачах и arraylimit="2" задачи разбиваются на три партии: 2, 2 и 1.

Поскольку нумерация каждого массива заданий начинается с 1, в программу добавлен NICA_ARRAY_OFFSET, который хранит начальную позицию текущей партии. Индекс задачи в полном массиве вычисляется следующим образом:

```
let "array_index = $SLURM_ARRAY_TASK_ID - 1 + ${NICA_ARRAY_OFFSET}"
```

Это позволяет разным партиям считывать правильные входные файлы и параметры задач и предотвращает повторное выполнение.

Что касается queuelimit, перед отправкой задачи Scheduler запрашивает количество заданий текущего пользователя в пакетной очереди. Если количество заданий достигло верхнего предела, выполняется ожидание и повторная проверка; после снижения количества ниже предела отправка продолжается. Кроме того, программа записывает номера заданий каждой партии, чтобы гарантировать, что последующие задачи или операции объединения результатов дожидаются завершения выполнения всех партий.

Тестирование:

Поскольку текущая тестовая среда не подключена к реальной кластерной системе пакетной обработки, невозможно фактически выполнить команды отправки, такие как sbatch и qsub, а также проверить реальное состояние очереди. Поэтому в данном тестировании используется метод статической проверки: путём проверки сгенерированных NICA-Scheduler скриптов заданий определяется, правильно ли реализована функциональность arraylimit. Полный тестовый XML-файл приведён в приложении E.

В тестовом файле задано 5 входных задач, `arraylimit` установлен в 2, а `queuelimit` — в 10. Согласно проекту, 5 задач должны быть разбиты на три партии, количество задач в каждой партии соответственно 2, 2 и 1, а соответствующие глобальные смещения — 0, 2 и 4.

После запуска NICA-Scheduler сначала проверяются сгенерированные файлы `job_*.sh`. Результаты показывают, что скрипт корректно сгенерировал массивы входных файлов, выходных файлов, начальных событий и количества событий, причём каждый массив содержит 5 наборов параметров задач, соответствующих конфигурации XML.

Затем проверяется логика вычисления индекса массива. В сгенерированный скрипт уже добавлен `NICA_ARRAY_OFFSET`, и при вычислении `array_index` учитывается смещение партии. Когда смещения трёх партий равны 0, 2 и 4, первая задача массива в каждой партии обращается соответственно к 1-му, 3-му и 5-му элементу полного массива задач. Таким образом, разные партии могут корректно считывать соответствующие параметры задач и не будут повторно обрабатывать входные файлы из предыдущих партий.

Результаты тестирования показывают, что NICA-Scheduler способен корректно разбирать параметры задач и генерировать Shell-скрипты, поддерживающие разбитые на партии массивы задач, а логика глобального смещения, необходимая для `arraylimit`, соответствует проектным требованиям.

```

(nica_env) chang@chang:~/bmn-container/opt/nica_scheduler_install$ cat job_88334861147373543.sh
#!/bin/bash

echo "Start date: `date` (host: `hostname`)"
echo "Current directory: `pwd`"

. ~/bmnroot/build/config.sh;

declare -a InputFileArray=("/tmp/task6_in1.root" "/tmp/task6_in2.root" "/tmp/task6_in3.root" "/tmp/task6_in4.root" "/tmp/task6_in5.ro
ot" )

declare -a OutputFileArray=("$VMCWORKDIR/macro/run/task6_out/out1.root" "$VMCWORKDIR/macro/run/task6_out/out2.root" "$VMCWORKDIR/macr
o/run/task6_out/out3.root" "$VMCWORKDIR/macro/run/task6_out/out4.root" "$VMCWORKDIR/macro/run/task6_out/out5.root" )

declare -a StartEventArray=("0" "10" "20" "30" "40" )

declare -a EventCountArray=("10" "10" "10" "10" "10" )

declare -a AddArgsArray(">" ">" ">" ">" ">" )

declare -a PutArray(">" ">" ">" ">" ">" )

declare -a GetArray(">" ">" ">" ">" ">" )

declare -a CleanArray=()

: ${NICA_ARRAY_OFFSET:=0}

let "array_index = $SLURM_ARRAY_TASK_ID - 1 + ${NICA_ARRAY_OFFSET} + 0"

```

Рисунок 3-11 – Массивы параметров и вычисление индекса с использованием NICA_ARRAY_OFFSET

Заключение

В данной работе реализованы шесть основных функций. Во-первых, обеспечено автоматическое создание выходного каталога, что позволяет избежать невозможности формирования файлов результатов из-за отсутствия целевого каталога. Во-вторых, расширена функция извлечения дополнительных файлов результатов: теперь система, помимо основных ROOT-файлов, может собирать журналы, отладочную информацию и другие вспомогательные результаты по имени файла или с использованием подстановочных знаков. В-третьих, добавлен способ пакетного ввода на основе JSON-файлов, благодаря чему система может напрямую читать список файлов в формате JSON и создавать соответствующую задачу обработки для каждого входного файла.

В части объединения частичных результатов в данной работе были протестированы и усовершенствованы существующие способы объединения, а также реализовано автоматическое определение имени дерева ROOT. Модифицированная система может находить фактически существующее дерево данных в файлах частичных результатов, проверять согласованность структуры деревьев в разных файлах и выполнять объединение с использованием правильного имени дерева, больше не полагаясь на фиксированное имя.

Для решения проблемы невозможности совместного выполнения обычных команд Shell и макросов ROOT в одной задаче в данной работе была реорганизована процедура разбора и выполнения задач, так что несколько команд и макросов могут выполняться последовательно в порядке их расположения в XML-файле. Результаты тестирования показывают, что в трёх режимах — local, global и container — система сохраняет правильный порядок выполнения и обеспечивает передачу файлов между различными этапами обработки.

Кроме того, в данной работе реализованы функции ограничения размера массива задач и количества задач в очереди. Когда число задач, подлежащих обработке, велико, система может разбивать крупные задачи на несколько меньших пакетов и гарантировать, что каждый пакет считывает правильные входные файлы и параметры событий; в процессе отправки задач также можно в зависимости от текущего количества задач в очереди решать, продолжать ли отправку, чтобы не превысить ограничения вычислительной платформы.

Результаты экспериментов показывают, что реализованные в данной работе функции расширяют возможности NICA-Scheduler в области управления каталогами, сбора файлов результатов, пакетного ввода, объединения результатов, организации задач и управления крупномасштабными задачами, сохраняя при этом совместимость с исходными способами конфигурации и процедурой выполнения. Поскольку текущая тестовая среда не подключена к реальной кластерной системе пакетной обработки, часть кластерных функций была проверена в основном путём генерации скриптов и статических проверок; их работа в реальной среде очереди всё ещё требует дальнейшего тестирования.

Список литературы

1. Эксперимент BM@N NICA и программное обеспечение BMNRoot [Электронный ресурс]. – URL: <https://bmn.jinr.int/> (дата обращения: 2026-02-11).
2. Руководство пользователя NICA-Scheduler [Электронный ресурс]. – URL: https://git.jinr.ru/nica-computing/nica_scheduler/-/blob/dev/README.md (дата обращения: 2026-02-27).
3. Примеры XML-описаний [Электронный ресурс]. – URL: https://git.jinr.ru/nica-computing/nica_scheduler/-/tree/dev/examples (дата обращения: 2026-03-07).
4. Использование контейнера BMNRoot [Электронный ресурс]. – URL: <https://bmn.jinr.int/software-installation/> (дата обращения: 2026-03-10).
5. Руководство BMNRoot [Электронный ресурс]. – URL: <https://bmn.jinr.int/plugin/BmnRoot.%20Start%20Guide.pdf> (дата обращения: 2026-03-23).
6. Data Analysis Framework ROOT [Электронный ресурс]. – URL: <https://root.cern.ch/> (дата обращения: 2026-04-10).
7. ROOT. TChain Class Reference [Электронный ресурс]. – URL: <https://root.cern.ch/doc/master/classTChain.html> (дата обращения: 2026-04-22).
8. FairRoot Framework Documentation [Электронный ресурс]. – URL: <https://fairrootgroup.github.io/FairRoot/> (дата обращения: 2026-04-30).
9. Slurm Workload Manager. Job Array Support [Электронный ресурс]. – URL: https://slurm.schedmd.com/job_array.html (дата обращения: 2026-05-03).
10. Apptainer User Guide. Bind Paths and Mounts [Электронный ресурс]. – URL: https://apptainer.org/docs/user/main/bind_paths_and_mounts.html (дата обращения: 2026-05-09).

11. W3C. Extensible Markup Language (XML) 1.0 (Fifth Edition) [Электронный ресурс]. – URL: <https://www.w3.org/TR/xml/> (дата обращения: 2026-05-15).
12. Bray T. The JavaScript Object Notation (JSON) Data Interchange Format. RFC 8259 [Электронный ресурс]. – URL: <https://www.rfc-editor.org/rfc/rfc8259.html> (дата обращения: 2026-05-21).
13. GNU Coreutils. mkdir: Make Directories [Электронный ресурс]. – URL: https://www.gnu.org/software/coreutils/manual/html_node/mkdir-invocation.html (дата обращения: 2026-06-01).
14. Gertsenberger K. V., Rogachevsky O. V. Current Workflow Execution Using Job Scheduling for the NICA Experiments // Proceedings of GRID 2018. – P. 134–138. – URL: <https://ceur-ws.org/Vol-2267/134-138-paper-24.pdf> (дата обращения: 2026-06-11).
15. Afanasiev S. et al. The BM@N Spectrometer at the NICA Accelerator Complex // Nuclear Instruments and Methods in Physics Research Section A. – 2024. – Vol. 1065. – Art. 169532. – DOI: 10.1016/j.nima.2024.169532. – URL: <https://doi.org/10.1016/j.nima.2024.169532> (дата обращения: 2026-06-15).

Приложение А

test_task2_extras.C

```
#include <TFile.h>
#include <TString.h>
#include <TSystem.h>
#include <TTree.h>

#include <algorithm>
#include <fstream>
#include <iostream>

void test_task2_extras(const char* inputFile,
                      const char* outputFile,
                      Int_t startEvent = 0,
                      Int_t eventCount = -1)
{
    TString inputPath = inputFile;
    TString outputPath = outputFile;
    gSystem->ExpandPathName(inputPath);
    gSystem->ExpandPathName(outputPath);

    TFile input(inputPath, "READ");
    TTree* inputTree = nullptr;
    input.GetObject("events", inputTree);
    if (!inputTree) {
        std::cerr << "ERROR: tree 'events' was not found" << std::endl;
        return;
    }

    TString outputDir = gSystem->DirName(outputPath.Data());
    gSystem->mkdir(outputDir, kTRUE);

    TFile output(outputPath, "RECREATE");
    TTree* outputTree = inputTree->CloneTree(0);

    Long64_t begin = startEvent < 0 ? 0 : startEvent;
    Long64_t end = inputTree->GetEntries();
    if (eventCount >= 0) {
        end = std::min(end, begin + static_cast<Long64_t>(eventCount));
    }

    for (Long64_t i = begin; i < end; ++i) {
        inputTree->GetEntry(i);
        outputTree->Fill();
    }

    outputTree->Write();
    output.Close();
    input.Close();

    std::ofstream log1(Form("%s/simulation.log", outputDir.Data()));
    log1 << "task2 extras test: main log" << std::endl;
    log1 << "processed events: " << (end - begin) << std::endl;

    std::ofstream log2(Form("%s/simulation_debug.log",
outputDir.Data()));
    log2 << "task2 extras test: debug log" << std::endl;
    log2 << "input: " << inputPath.Data() << std::endl;
```

```

    log2 << "output: " << outputPath.Data() << std::endl;
}

```

task2_extras_test.xml

```

<job name="task2_extras_test">
  <macro path="$VMCWORKDIR/macro/run/test_task2_extras.C">
    <file input="$VMCWORKDIR/macro/run/task2_test/input.root"
          output="$VMCWORKDIR/macro/run/task2_test/final/result.root"
          start_event="0"
          event_count="5">
      <get command="cp"
            directory="/opt/bmnroot/macro/run/nica_tmp_get"
            extras="simulation*.log"/>
    </file>
  </macro>
  <run mode="local"
        count="1"
        config="$~/bmnroot/build/config.sh"/>
</job>

```

Приложение В

```

<job name="test_json_input_local">
  <macro path="$VMCWORKDIR/macro/run/test_json_input.C"
        start_event="10" event_count="20">
    <file
      json_input="$VMCWORKDIR/macro/run/test_json_input/inputs.json"
      output="$VMCWORKDIR/macro/run/out_${counter}.root"/>
    </macro>

    <run mode="local"
          count="2"
          config="$~/bmnroot/build/config.sh"
          logs="$VMCWORKDIR/macro/run/local_log"/>
</job>

```

Приложение С

create_test_tree.C

```

#include <TFile.h>

#include <TTree.h>
#include <iostream>

void create_test_tree(
    const char* fileName,
    const char* treeName,
    int startEvent,
    int eventCount)
{
    TFile file(fileName, "RECREATE");

    if (file.IsZombie())
    {
        std::cerr
            << "ERROR: cannot create file: "

```

```

        << fileName
        << std::endl;
    return;
}

TTree tree(treeName, treeName);

int event_id = 0;
tree.Branch(
    "event_id",
    &event_id,
    "event_id/I"
);

for (int i = 0; i < eventCount; ++i)
{
    event_id = startEvent + i;
    tree.Fill();
}

tree.Write();
file.Close();

std::cout
    << "Created file: " << fileName
    << ", tree: " << treeName
    << ", entries: " << eventCount
    << std::endl;
}

```

Приложение D

task5_macro1.C

```

#include <fstream>
#include "TFile.h"
#include "TTree.h"
#include "TString.h"
#include "TSystem.h"

void task5_macro1(const char* inputFile,
                  const char* outputFile,
                  int startEvent = 0,
                  int eventCount = 0)
{
    TString orderPath = TString(gSystem->Getenv("VMCWORKDIR")) +
"/macro/run/task5_test/order.txt";

    std::ofstream order(orderPath.Data(), std::ios::app);
    order << "MACRO_1" << std::endl;
    order.close();

    TFile f(outputFile, "RECREATE");
    TTree tree("task5tree1", "task5 test tree 1");

    int start = startEvent;
    int count = eventCount;

    tree.Branch("start", &start, "start/I");
    tree.Branch("count", &count, "count/I");
}

```

```

        tree.Fill();
        tree.Write();
        f.Close();
    }

```

task5_macro2.C

```

#include <fstream>
#include "TFile.h"
#include "TTree.h"
#include "TString.h"
#include "TSystem.h"

void task5_macro2(const char* inputFile,
                  const char* outputFile,
                  int startEvent = 0,
                  int eventCount = 0)
{
    TString orderPath = TString(gSystem->Getenv("VMCWORKDIR")) +
"/macro/run/task5_test/order.txt";

    std::ofstream order(orderPath.Data(), std::ios::app);
    order << "MACRO_2" << std::endl;
    order.close();

    TFile f(outputFile, "RECREATE");
    TTree tree("task5tree2", "task5 test tree 2");

    int start = startEvent;
    int count = eventCount;

    tree.Branch("start", &start, "start/I");
    tree.Branch("count", &count, "count/I");

    tree.Fill();
    tree.Write();
    f.Close();
}

```

test_task5_stack.xml

```

<job name="test_task5_stack">
  <command line="echo COMMAND_1 >
$VMCWORKDIR/macro/run/task5_test/order.txt"/>

  <macro path="$VMCWORKDIR/macro/run/task5_test/task5_macro1.C">
    <file input="$VMCWORKDIR/macro/run/task5_test/input.root"
          output="$VMCWORKDIR/macro/run/task5_test/out_macro1.root"
          start_event="0"
          event_count="10"/>
  </macro>

  <command line="echo COMMAND_2 >>
$VMCWORKDIR/macro/run/task5_test/order.txt"/>

  <macro path="$VMCWORKDIR/macro/run/task5_test/task5_macro2.C">
    <file input="$VMCWORKDIR/macro/run/task5_test/out_macro1.root"
          output="$VMCWORKDIR/macro/run/task5_test/out_macro2.root"
          start_event="10"
          event_count="20"/>
  </macro>

```

```

    <command line="echo COMMAND_3 >>
$VMCWORKDIR/macro/run/task5_test/order.txt"/>

    <run mode="local" count="1" config="~/bmnroot/build/config.sh"/>
</job>

```

Приложение Е

```

<job name="test_task6_arraylimit">
  <macro path="$VMCWORKDIR/macro/run/test_task6.C">
    <file input="/tmp/task6_in1.root"
output="$VMCWORKDIR/macro/run/task6_out/out1.root" start_event="0"
event_count="10"/>
    <file input="/tmp/task6_in2.root"
output="$VMCWORKDIR/macro/run/task6_out/out2.root" start_event="10"
event_count="10"/>
    <file input="/tmp/task6_in3.root"
output="$VMCWORKDIR/macro/run/task6_out/out3.root" start_event="20"
event_count="10"/>
    <file input="/tmp/task6_in4.root"
output="$VMCWORKDIR/macro/run/task6_out/out4.root" start_event="30"
event_count="10"/>
    <file input="/tmp/task6_in5.root"
output="$VMCWORKDIR/macro/run/task6_out/out5.root" start_event="40"
event_count="10"/>
  </macro>

  <run mode="global"
    count="1"
    arraylimit="2"
    queuelimit="10"
    config="~/bmnroot/build/config.sh"/>
</job>

```